As shown in figure 6.2, the block header, Tx tree, and receipt tree are *per-block* data structures. That characterization means that for every new block added to the chain, there is a new instance of the block header, a new Tx tree, and a new set of receipts.

On the other hand, the state tree is a *per-blockchain* data structure: it stores the current state of all the accounts on the blockchain, starting from the genesis block. The state tree records the history of what happened on the blockchain as a whole and how it happened. Its state keeps changing as affected by the Txs executed. The state information is valuable information for verifying that certain Txs took place and for mining (or searching) for particular actions, changes, and events. You can analyze the on-chain data stored over a range of blocks to identify a pattern in transactions issued for a specific smart contract, for example.

I'm sure that you're eager to start coding, but a good understanding of these elements will help you design better Dapps. As you'll recall from chapter 3, the blockchain is not your regular database, and you'll want to keep only essential data in there. The information provided in this chapter will help you organize your on-chain data by designing the Txs, receipts, and state variables of your applications so as not to overload the chain. This knowledge will also be useful in extracting on-chain data records for offline analytics and decision-making.

6.2 Blind auction use case

The four items we've just examined--block header, Txs, receipts, and state storage--form the majority of the on-chain data. This data plays a significant role in ensuring the robustness and